Project 11 — Variational Autoencoder (VAE)

Level: Advanced | **Dataset:** MNIST (torchvision) | **Framework:** PyTorch

---

Objective

Build a VAE to learn a continuous, structured latent space and generate new digits.
Cover: reparameterization trick, ELBO loss (reconstruction + KL divergence), latent space interpolation, conditional generation.

Project Structure

```
11_vae_mnist/
├── notebooks/
│   ├── 01_vae_theory.ipynb
│   ├── 02_train.ipynb
│   └── 03_latent_explore.ipynb
├── data/
├── models/model.pkl
├── charts/
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Notebook 01 — Theory (`01_vae_theory.ipynb`)

STOP 1 — AE vs VAE Core Difference

Write theory cells explaining:

- AE: $x \to z$ (point) $\to \hat{x}$ — deterministic latent space
- VAE: $x \to (\mu, \sigma) \to z \sim N(\mu, \sigma^2) \to \hat{x}$ — stochastic latent space
- Run a simple AE on 2D toy data, show the disconnected latent space
- **Agent stops here. Explain:**
  - Why AE's latent space has "holes": decoder was never trained on points between training samples
  - What "holes" cause: generating from the middle of latent space gives nonsense
  - How VAE fixes this: forces latent space to be a continuous smooth Gaussian
  - The key intuition: VAE learns WHERE to put things + HOW WIDE to make the region around them
- Wait for user confirmation before continuing

STOP 2 — Probabilistic Encoder

- In a VAE, the encoder outputs TWO vectors: `mu` and `log_var` (each shape [B, latent_dim])
- From these, we sample: `z = mu + epsilon * std` where `epsilon ~ N(0,1)` and `std = exp(0.5 * log_var)`
- **Agent stops here. Explain:**
  - Why we output log_var not var: log_var can be any real number, var must be positive
  - What the encoder is learning: a distribution over z, not a single point
  - The sampling process: every forward pass samples a different z (stochastic)
  - Why this stochasticity enables generation: we can sample from N(0,1) without needing an input
- Wait for confirmation

STOP 3 — Reparameterization Trick

Write math cells:

- Naive: $z \sim N(\mu, \sigma^2)$ — cannot backpropagate through sampling (stochastic node)
- Reparameterized: $z = \mu + \sigma * \varepsilon$, where $\varepsilon \sim N(0,1)$ — gradients flow through μ and σ
- Implement both and show that naive breaks `.backward()`
- **Agent stops here. Explain:**
  - Why we can't backpropagate through a sampling operation (not a deterministic function)
  - The trick: move the randomness to ε (a separate input), make z a DETERMINISTIC function of $(\mu, \sigma, \varepsilon)$
  - Why gradients now flow through μ and σ: they're just parameters in $z = \mu + \sigma * \varepsilon$
  - This is one of the most important tricks in modern deep learning
- Wait for confirmation

Notebook 02 — Training (`02_train.ipynb`)

STOP 4 — VAE Architecture

```
Encoder:
  Flatten (28*28=784) → Linear(784, 400) → ReLU
  → Linear(400, latent_dim*2) split into → mu [B, latent_dim], log_var [B,
latent_dim]

Reparameterize: z = mu + exp(0.5*log_var) * epsilon

Decoder:
  Linear(latent_dim, 400) → ReLU
  Linear(400, 784) → Sigmoid [output in (0,1) — pixel values]
```

- Use `latent_dim=20`
- **Agent stops here. Explain:**
  - Why Sigmoid at decoder output: MNIST pixels in [0,1]
  - Why latent_dim=20: enough to encode digit identity + style
  - How to split encoder output into mu and log_var: `mu, log_var = out.chunk(2, dim=1)` or `out[:, :ld]` and `out[:, ld:]`
  - What the 20-dimensional z represents: each dimension captures some aspect of digit variation
- Wait for confirmation

STOP 5 — ELBO Loss Function

```python
def elbo_loss(x, x_hat, mu, log_var, beta=1.0):
    # Reconstruction: binary cross entropy (pixels are in [0,1])
    recon = F.binary_cross_entropy(x_hat, x, reduction='sum')
    # KL divergence: push posterior N(mu, sigma) toward prior N(0,1)
    kl = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return (recon + beta * kl) / x.size(0)  # normalize by batch size
```

- **Agent stops here. Explain:**
  - What ELBO means: Evidence Lower BOund — maximizing ELBO ≈ maximizing likelihood
  - The two terms:
    1. Reconstruction loss: AE objective — how well we reconstruct input
    2. KL divergence: regularizer — pushes z distribution toward standard Gaussian
  - Why KL toward N(0,1): so we can sample from N(0,1) at generation time
  - The KL formula: -0.5 * sum(1 + log_var - mu² - exp(log_var)) — derive this
  - What β does (β-VAE): β>1 encourages more disentangled latent space
- Wait for confirmation

STOP 6 — KL Annealing

- Start with β=0, linearly increase to β=1 over first 20 epochs
- Plot reconstruction loss and KL loss separately per epoch
- **Agent stops here. Explain:**
  - What KL annealing solves: "posterior collapse" — without annealing, KL often collapses to 0 early
  - Posterior collapse: model ignores z and decoder learns to reconstruct without latent code
  - Why starting with β=0 allows the model to learn reconstruction first
  - How to detect posterior collapse: KL term goes to 0, mu and log_var stop changing
- Wait for confirmation

STOP 7 — Training Loop

- 50 epochs, Adam lr=1e-3, batch_size=128
- Track total ELBO, reconstruction term, KL term separately
- Plot all three curves
- **Agent stops here. Explain:**
  - Why tracking reconstruction and KL separately is important (not just total loss)
  - What healthy training looks like: KL increases gradually, reconstruction decreases
  - What unhealthy training looks like: KL → 0 (collapse) or reconstruction doesn't decrease
- Wait for confirmation

Notebook 03 — Latent Space Exploration (`03_latent_explore.ipynb`)

STOP 8 — 2D Latent Space Visualization

- If latent_dim=2 (train a separate 2D version): plot all test digits in 2D z-space colored by digit label
- If latent_dim=20: use t-SNE to project to 2D
- **Agent stops here. Explain:**
  - What we expect: each digit forms a cluster, similar digits (4 vs 9, 3 vs 8) cluster closer
  - What "disentangled" means: different dimensions control different factors (style, rotation, thickness)
  - How the VAE latent space is structured compared to regular AE (smooth, no holes)
- Wait for confirmation

STOP 9 — Latent Space Interpolation

- Encode two different digit images to get z1, z2
- Generate 10 intermediate z values: `z = (1-t)*z1 + t*z2` for t in [0,1]
- Decode each z and display as a row of images
- **Agent stops here. Explain:**
  - Why interpolation works in VAE but not in AE: VAE latent space is continuous (no holes)
  - What a smooth interpolation shows: gradual morphing from digit A to digit B
  - What a broken AE interpolation shows: sudden jumps and nonsense images in the middle
  - This is the key visual proof that VAE learns a better structured latent space
- Wait for confirmation

STOP 10 — Generation from Prior

- Sample 64 z vectors from N(0,1): `z = torch.randn(64, latent_dim)`
- Decode all 64 samples
- Display as 8×8 grid of generated digits
- **Agent stops here. Explain:**
  - Why sampling from N(0,1) works: KL loss forced the posterior to be close to N(0,1)
  - What good generation looks like: recognizable digits, diverse styles
  - What bad generation looks like (poor training or posterior collapse): blurry identical images
  - The fundamental difference from AE: AE cannot generate because we don't know which z values are valid
- Wait for confirmation

STOP 11 — Digit-Conditioned Generation (Simple)

- For each of 10 digit classes: find all test samples, average their z vectors → class prototype z
- Generate 10 images from the 10 prototype z vectors
- **Agent stops here. Explain:**
  - What "class prototype in latent space" means: center of the class cluster
  - How to do true conditional generation (Conditional VAE — CVAE): feed label as input to encoder and decoder
  - Why this simple approach works at all: VAE clusters same-class digits together in z-space
- Wait for confirmation

STOP 12 — Reconstruction Quality

- Pick 20 test images
- Show side by side: original | reconstruction
- Compute SSIM (Structural Similarity) between originals and reconstructions
- **Agent stops here. Explain:**
  - Why reconstructions look slightly blurry: VAE averages over the distribution → blurriness
  - The VAE-GAN tradeoff: VAE → blurry but stable, GAN → sharp but training unstable
  - What SSIM measures vs MSE: SSIM accounts for structure, not just pixel values
- Wait for confirmation

STOP 13 — Save & Generation Function

- Save model.state_dict()
- Write `generate(n=16)` → n images sampled from prior
- Write `encode_and_reconstruct(pil_image)` → z_vector, reconstructed_image
- Write `interpolate(img1, img2, steps=10)` → list of 10 images
- **Agent stops here. Explain:**
  - Why generation requires no input (unlike all previous projects)
  - The three use modes of a trained VAE: reconstruct, encode, generate
  - Which operations require `torch.no_grad()` and which don't (generation always needs it)
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model()` → model
- `generate_digits(n=16)` → grid image
- `interpolate(z1, z2, steps=10)` → list of PIL images
- `get_latent_viz()` → 2D coords + labels for all test digits
- `get_training_curves()` → recon_loss, kl_loss, total_loss arrays

`app.py` — Streamlit (~80 lines)

Sections:

1. "Generate" button → display 8×8 grid of generated digits
2. Upload digit image → show reconstruction + z vector values
3. Tab 1: ELBO curve (split into recon + KL)
4. Tab 2: t-SNE latent space scatter plot colored by digit
5. Tab 3: Interpolation visualization between two selected digits

---

Key Concepts Covered

- AE vs VAE: deterministic vs stochastic latent space
- Reparameterization trick (the core DL trick)
- ELBO loss = reconstruction + β * KL divergence
- KL divergence math: -0.5 * sum(1 + log_var - mu^2 - exp(log_var))
- Posterior collapse and KL annealing
- Latent space interpolation (proof of continuity)
- Generation from N(0,1) prior
- β-VAE for disentanglement